

Soft Chance-Constrained MPC for FSA-v5: Implementation Note

Gemini CLI Implementation Team

May 11, 2026

1 Overview

This note documents the implementation of soft chance constraints for the Fitness-Stress-Autonomic (FSA) version 5 model predictive control (MPC) framework. We have extended the existing Alertness (A) constraint to include Base Aerobic fitness (B) and Strength capacity (S), ensuring that the optimized training plans remain within safe physiological boundaries for all key state variables.

2 Mathematical Formulation

The controller optimizes the bimodal stimulus schedule $\Phi(t) = [\Phi_B(t), \Phi_S(t)]$ by minimizing a multi-objective cost function $J(\theta)$.

2.1 Objective Function

The unified cost functional J is defined as:

$$J = \lambda_\Phi J_{effort} + J_{reward} + \sum_{X \in \{A, B, S\}} \lambda_{chance, X} J_{chance, X} \quad (1)$$

2.2 Reward Term

To encourage balanced adaptation across all channels, the reward term now integrates the total area under the A , B , and S curves:

$$J_{reward} = - \int_0^T [A(t) + B(t) + S(t)] dt \quad (2)$$

2.3 Soft Chance Constraints

For each variable $X \in \{A, B, S\}$, we implement a differentiable soft penalty that activates when X drops below a specified threshold X_{thr} . We use a sigmoid relaxation σ multiplied by the quadratic violation depth, so that the penalty grows the further X drops below the threshold. The product remains smooth everywhere, which preserves compatibility with Hamiltonian Monte Carlo (HMC) gradients:

$$J_{chance, X} = \int_0^T \sigma \left(\beta \cdot \frac{X_{thr} - X(t)}{scale} \right) \cdot (X_{thr} - X(t))^2 dt \quad (3)$$

where:

- $\beta = 50.0$ (steepness of the smooth gate)
- $scale = 0.10$ (gate transition width)

Why the quadratic factor. An earlier version used the bare sigmoid $\sigma(\beta(X_{thr} - X)/scale)$ as the integrand. That form is bounded in $[0, 1]$, so once X drops more than a few multiples of $scale/\beta$ below the threshold the integrand saturates at 1 and stops growing: a trajectory at $X = X_{thr}/2$ incurs the same penalty as a trajectory at $X = 0$. This gave the controller no incentive to recover once it had fallen well below the threshold. Multiplying by $(X_{thr} - X)^2$ removes the saturation: the integrand is now $\approx 1 \cdot (X_{thr} - X)^2$ for $X \ll X_{thr}$ (quadratic in violation depth) and ≈ 0 for $X > X_{thr}$, while remaining C^∞ everywhere. Penalty values at probe $X = 0.05$:

- $X_{thr} = 0.1 \Rightarrow \sigma(\cdot) \cdot (0.05)^2 \approx 0.0025$
- $X_{thr} = 0.3 \Rightarrow \sigma(\cdot) \cdot (0.25)^2 \approx 0.0625$ ($25\times$ larger)

See `chance_penalty_options.png` in this folder for the comparison plot of the four candidate shapes considered.

Note on $\lambda_{chance,X}$ scale. The natural scale of the penalty weight has changed: the bounded-sigmoid form had units $[\text{prob}] \cdot [\text{time}]$, whereas the new form has units $[\text{depth}^2] \cdot [\text{time}]$. CLI defaults for $\lambda_{chance,X}$ may need re-tuning relative to the bounded-sigmoid era.

3 Implementation Details

3.1 Julia GPU Kernel

The cost accumulation logic in `gpu_control_v5.jl` was updated to compute three independent chance accumulators. The kernel now accepts the following parameters via the `FSAv5ControlGPUPerformanceTarget` structure:

- `A_thr`, `B_thr`, `S_thr`: Thresholds for each state.
- `lam_chance`, `lam_chance_B`, `lam_chance_S`: Respective penalty weights.

3.2 Lean4 Reference

The formal specification in `Cost.lean` was updated with the `softChancePenalty` function, maintaining the "Lean-first" philosophy where the Lean code serves as the executable ground truth for the differentiable surrogates.

4 CLI Interface

Five new Command Line Interface (CLI) arguments were added to `bench_args.jl` to allow for flexible configuration of the safety boundaries:

Argument	Default	Description
<code>-ctrl-a-thr</code>	0.3	Alertness safety threshold
<code>-ctrl-b-thr</code>	0.2	Base Aerobic safety threshold
<code>-ctrl-s-thr</code>	0.2	Strength safety threshold
<code>-ctrl-lam-chance-b</code>	1.0	Penalty weight for B violations
<code>-ctrl-lam-chance-s</code>	1.0	Penalty weight for S violations

Table 1: New CLI arguments for FSA-v5 Bench.

5 Verification

The implementation was originally verified using the expanded differential test suite in `test_lean_diff_v5.jl`. A total of 429 tests (including the `softChancePenalty` testset and updated single-thread GPU-CPU parity checks) passed successfully under the bounded-sigmoid form, confirming bit-equivalence between the Julia GPU implementation and the Lean4 functional reference.

The change to $\sigma \cdot (X_{thr} - X)^2$ updates the body of `softChancePenalty` in both Lean (`Cost.lean`) and the kernel comparison expression in the diff-test, but does not change the function signature or the GPU-CPU parity testset (which sets $\lambda_{chance,X} = 0$). The diff test must be re-run after this change to confirm bit-equivalence is preserved.